

BOUNCE · SECURITY REVIEW

Bounce Updates Review.

A security review of Updates Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Bounce	Updates Review	Guardian	October 6 – October 17, 2025	October 24, 2025

§ 00

Contents.

30 pages — overview, confidence ranking, scope, 20 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 11 findings	P. 09
	Remediation Review · 9 findings	P. 20
§ 05	Disclaimer	P. 29
§ 06	About Guardian	P. 30

§ 01

Overview.

Guardian reviewed Updates Review across 2 review rounds. The final report includes 3 High, 4 Medium, 3 Low, 10 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	bounce-tech/bounce-contracts
CLIENT	METHODOLOGY
Bounce	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Updates Review	605015dde32c
PREPARED BY	REMEDATION REVIEW COMMIT
Guardian	4016d3a5fb13
AUDIT WINDOW	
October 6 – October 17, 2025	
FINAL REPORT	
October 24, 2025	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	3	0	3	0	0
Medium	4	0	4	0	0
Low	3	0	3	0	0
Informational	10	0	10	0	0
All severities	20	0	20	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Updates Review in `bounce-tech/bounce-contracts`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Referral Rebates Credited To Agent Wallet	Logical Error	HIGH	ACKNOWLEDGED	p. 9
M-01	Redeemers capture post request profit	Logical Error	MEDIUM	ACKNOWLEDGED	p. 10
M-02	Inability To Deregister API Wallets From Core	Logical Error	MEDIUM	ACKNOWLEDGED	p. 11
M-03	Adding Agent May Silently Fail	Logical Error	MEDIUM	ACKNOWLEDGED	p. 12
L-01	Superfluous Storage Read	Superfluous Code	LOW	ACKNOWLEDGED	p. 13
L-02	Missing Execution Redemption Fee Validation	Configuration	LOW	ACKNOWLEDGED	p. 14
L-03	Unnecessary Totaly Supply Check	Gas Optimization	LOW	ACKNOWLEDGED	p. 15
I-01	Redundant balance check in executeRedemptions	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 16
I-02	Redundant User Credit Check	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 17
I-03	Unused error	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 18
I-04	Misleading Function Param Name	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 19
REMEDIATION REVIEW					
H-01	Protocol Wide Deadlock	DoS	HIGH	ACKNOWLEDGED	p. 20

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-02	Agent Wallets Spot Transfers Not Supported	Logical Error	HIGH	ACKNOWLEDGED	p. 21
M-01	Unnamed agent can't be set	Configuration	MEDIUM	ACKNOWLEDGED	p. 22
I-01	Magic Values For Agent Slots	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 23
I-02	Superfluous Validations	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 24
I-03	Leverage Truncated To Integer	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 25
I-04	Unused Events	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 26
I-05	Referee Typo	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 27
I-06	Leveraged Token Setup In HyperCore	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 28

HIGH **H-01**

ACKNOWLEDGED

Referral Rebates Credited To Agent Wallet

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L353
REVIEW	Main Review

DESCRIPTION

When agents execute prepared redemptions, the contract calls `donateRebates` passing `msg.sender` as the user. In the `executeRedemptions` path, `msg.sender` is the agent, not the end-user whose redemption is being processed. As a result, referral rebates are attributed to the agent rather than the redeemer, diverting rebates to agents and denying them to users.

RECOMMENDATION

In the `executeRedemptions` flow, pass the current user address being processed to `donateRebates` instead of `msg.sender`.

As the `redeem` is called by user, update `payRedemptionFee` to accept a beneficiary argument and, in `executeRedemptions`, call `donateRebates(user, redemptionFee_)` rather than `donateRebates(msg.sender, redemptionFee_)`.

RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/106>

MEDIUM **M-01**

ACKNOWLEDGED

Redeemers capture post request profit

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L208
REVIEW	Main Review

— DESCRIPTION

When users want to redeem their LT tokens, they have two paths: through direct redemption if the contract holds enough assets, or through a two-step redemption if the contract does not hold enough assets. In the latter case, users submit a redemption request, and agents later bridge funds from Core to the EVM to fulfill it.

When users request redemption, they effectively give up their LT tokens, and the amount of assets they should receive upon fulfillment is the same as what they would have received if the redemption were executed immediately at the time of the request (assuming no loss occurs).

However, in the current implementation, within `executeRedemptions`, each withdrawal request is processed in sequence, and the amount of assets is calculated based on the current exchange rate, which may include profits that accrued after the request was made. This means requesters can receive a portion of profits generated by other users after their redemption request.

— RECOMMENDATION

Consider capping the redemption amount to what the user would have received if the request were fulfilled immediately at the time of request, preventing users from capturing profits that accrued after their redemption was made.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Inability To Deregister API Wallets From Core

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L269
REVIEW	Main Review

DESCRIPTION

Leveraged token owner can add agent wallets, by calling `CoreWriterLib.addApiWallet` with agent address and name. It also stores the agent wallet in EVM state.

However, `removeAgent` only removes agent from EVM side, but will remain registered in HyperCore and continue to trade on behalf of the LeveragedToken contract. Be aware that agents are not exactly removed on hyperliquid, only replaced (named and unnamed), with a default expiration time of 180 days.

In case there is a malicious agent detected, owner can only prevent EVM interactions, but won't be able to stop Core trading.

RECOMMENDATION

Make sure to call `CoreWriterLib.addApiWallet` with agent address and the same agent name used during the `addAgent` flow.

Be aware that if the api wallet already expired, calling `CoreWriterLib.addApiWallet` will register again, so a call to HL API `{"type":"extraAgents","user":"0x**"}` must be used to verify if the agent expired, so it's not re-added.

Alternatively, add a `onlyOwner` function that only calls `CoreWriterLib.addApiWallet` to replace agent wallets in Core.

RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/112>

MEDIUM **M-03**

ACKNOWLEDGED

Adding Agent May Silently Fail

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L269
REVIEW	Main Review

DESCRIPTION

The `addAgent` function will execute `CoreWriterLib.addApiWallet` to add a new agent address in HyperCore. However, the EVM transaction will always succeed even if agent is never added in core. This may happen if:

- There are more than 3 named agents. According to HL docs `An account can have 1 unnamed approved wallet and up to 3 named ones`. Therefore, the 4th named agent will never be added to core.
- Agent wallet is already an agent for another user
- Agent wallet is an active HL user.

RECOMMENDATION

Consider refactoring the agent logic, merging both `addAgent` and `removeAgent` in one single owner function, passing the agent address and index. If index is 0, it will replace the unnamed agent, else it will calculate the agent name with given index.

Storage wise, this implementation will only require the `isAgent` (to verify `onlyAgents` access control) and `agentCreatedAt` (to prevent reusing an agent wallet).

Additionally, double check the agent being added, and use the API `/info` endpoint with `{"type":"extraAgents", "user":"<ltAddress>"}` to confirm and `{"type":"userRole", "user":"<agentToAdd>"}` to validate the agent availability.

RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/112>

LOW **L-01**

ACKNOWLEDGED

Superfluous Storage Read

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L64
REVIEW	Main Review

DESCRIPTION

The `whenNotMintPaused` modifier fetches contract storage but the storage variable `$` is never used.

RECOMMENDATION

Remove the storage read.

RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/111>

LOW **L-02**

ACKNOWLEDGED

Missing Execution Redemption Fee Validation

CATEGORY	Configuration
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L214
REVIEW	Main Review

DESCRIPTION

The execution redemption fee is a flat fee added to the two step redemption flow to cover extra gas costs involved to bring back funds to HyperEVM.

Protocol owner uses `setExecuteRedemptionFee` to set this fee. However, there is no max value enforced, specially comparing it to the minimum transaction size, so redemptions will not be executed as `baseAmount_ < redemptionFee_`.

RECOMMENDATION

Consider enforcing the new `setExecuteRedemptionFee` value is not above (or even close) to the min transaction size.

RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/110>

LOW **L-03**

ACKNOWLEDGED

Unnecessary Totaly Supply Check

CATEGORY	Gas Optimization
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L294
REVIEW	Main Review

— DESCRIPTION

The `exchangeRate` function calculates the current ratio between assets and LT supply. If `totalSupply` is 0, then the exchange rate is hardcoded to 1e18, to avoid division by zero.

However, the current leveraged token deployment will mint a minimum amount and sends LTs to the dead address. Therefore, `totalSupply` will never be 0 after this initial mint and it will only waste gas for users.

— RECOMMENDATION

Remove the zero total supply check. To mint the initial protocol supply, use the initialize function instead with a simpler mint flow that calls `_checkpoint` and mints the LTs 1:1 to the dead address.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Redundant balance check in executeRedemptions

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L215>

REVIEW Main Review

— DESCRIPTION

Inside `executeRedemptions`, the contract first verifies that the available balance covers the full redemption amount:

```
if (baseAssetBalance() < baseAmount_) continue;
```

However, a few lines later, it performs another check:

```
if (baseAsset().balanceOf(address(this)) < afterFees) continue;
```

This second condition is redundant, since `afterFees_` is always less than `baseAmount_`. If the first check passes, the second will necessarily pass as well, making it unnecessary.

— RECOMMENDATION

Remove the redundant balance check as keeping only the `baseAssetBalance() < baseAmount_` condition is sufficient to ensure redemption safety.

— RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/107>

INFORMATIONAL **I-02**

ACKNOWLEDGED

Redundant User Credit Check

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L211
REVIEW	Main Review

DESCRIPTION

During `executeRedemptions`, the function first fetches user credit and assigns to `ltAmount_`:

```
uint256 ltAmount_ = userCredit(user_);
```

However, it later compares the user credit to the `ltAmount_`:

```
if ($.userCredit[user_] < ltAmount_) continue;
```

This extra check is redundant, as the `ltAmount_ == $.userCredit[user_]`

RECOMMENDATION

Remove the following check:

```
if ($.userCredit[user_] < ltAmount_) continue;
```

RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/109>

INFORMATIONAL **I-03**

ACKNOWLEDGED

Unused error

CATEGORY	Best Practices
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/interfaces/ILeveragedToken.sol#L16
REVIEW	Main Review

— DESCRIPTION

Unused `NotEnoughBaseAsset` error in `src/interfaces/ILeveragedToken.sol`.

— RECOMMENDATION

Remove the unused error.

— RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/108>

INFORMATIONAL **I-04**

ACKNOWLEDGED

Misleading Function Param Name

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L344-L372
REVIEW	Main Review

DESCRIPTION

The `_baseAmount` param is normally used as the base asset amount, like in the `mint` function or the assigned to the returned value of `ltToBaseAmount`. The contract will later use this base amount to calculate fees.

However, the same param name is used in the fee payment functions. Although the amount is in base asset terms, it may lead to errors if its treated as the user amount instead of fees.

RECOMMENDATION

Consider changing the param name to differentiate with user base asset amounts and fee amounts.

RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/114>

HIGH **H-01**

ACKNOWLEDGED

Protocol Wide Deadlock

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L303
REVIEW	Remediation Review

DESCRIPTION

The `_checkpoint` function unconditionally computes and pays streaming fees from the on-chain `baseAsset` via `_payFees`. If the on-chain `baseAsset` balance is insufficient (because most AUM sits on Hyperliquid), fee transfers revert.

Since `bridgeIn` also calls `_checkpoint` first, agents cannot replenish the on-chain balance to make fee payment succeed. This creates a protocol-wide deadlock: mint, redeem (both paths), `executeRedemptions`, `bridgeIn/out`, and `checkpoint` all revert until someone donates `baseAsset` directly to the contract.

Although team already mentioned that they will keep 10% assets in the `LeveragedToken` contract, a malicious user can redeem the exact amount of LT tokens and leave the contract empty, DoS'ing next calls to `_checkpoint`.

RECOMMENDATION

Consider adding a max redeem function, to prevent users from redeeming 100% of the base asset balance in the contract.

PROOF OF CONCEPT <https://gist.github.com/wafflemakr/13d1a36a39e69789ee8e74e35f478499>

RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/124>

HIGH **H-02**

ACKNOWLEDGED

Agent Wallets Spot Transfers Not Supported

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol
REVIEW	Remediation Review

— DESCRIPTION

Agent wallets will be able to bridge funds in/out of HyperEVM, execute redemptions and manually perform a checkpoint. On the HyperCore side, these agents will swap base assets to USDC, transfer between spot and perps, and open positions for the leveraged token contract.

However, agent wallets are not allowed to perform `usd_class_transfer` actions, to transfer USDC between spot and perps, as they are meant to be used only to trigger order actions on behalf of the main user.

Due to the fact that funds can't never reach the perps balance, the `LeveragedToken` lacks the functionality to open perpetual positions. Spot transfer are meant to be called by the main account, in this case the `LeveragedToken`, but there is no permissioned function to trigger this action from the EVM side.

— RECOMMENDATION

Consider adding a `onlyAgents` protected function that allows these wallets to perform `usd_class_transfer` actions on behalf of the parent account:

```
CoreWriterLib.transferUsdClass(amount_, toPerp_);
```

Keep in mind that `amount_` has EVM units, so there is no need to do any conversion.

PROOF OF CONCEPT https://github.com/hyperliquid-dex/hyperliquid-python-sdk/blob/master/examples/basic_spot_transfer.py#L10

— RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/115>

MEDIUM **M-01**

ACKNOWLEDGED

Unnamed agent can't be set

CATEGORY	Configuration
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L252
REVIEW	Remediation Review

DESCRIPTION

`setAgent` allows the owner to set agents for the LT contract, it allows to set 3 agents from index 0 to 2, each has a derived name of `symbol_AGENT_idx`. On the other hand, HyperLiquid allows the addition of an unnamed agent which acts as the "default" agent. With the current implementation, it is not possible to set an unnamed agent using `setAgent` function as it always sets the agent name to `symbol_AGENT_idx`. This limits the LT contract to only have 3 agents, and the unnamed agent can't be set.

RECOMMENDATION

Consider refactoring the `setAgent` function to have a "reserved" index for the unnamed agent, for example, using index `0` for the unnamed agent and allowing named agents to be set from index `1` to `3`. This way, the LT contract can accommodate both named and unnamed agents effectively.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Magic Values For Agent Slots

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/interfaces/ILeveragedToken.sol#L56>

REVIEW Remediation Review

DESCRIPTION

The `_AGENT_SLOTS` is declared as a constant with a value of 3. However, this constant is not used in other parts of the code and instead uses the hardcoded integer 3 (i.e. `address[3] memory` or `address[3] agents`).

RECOMMENDATION

Consider using the constant instead of magic values.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Superfluous Validations

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol#L362
REVIEW	Remediation Review

DESCRIPTION

The `_addCredit` function validates that the amount is greater than zero. However, this function is only called by `prepareRedeem`, which already validates non zero `ltAmount_`, so `_addCredit` validation is not necessary. Additionally, `_addCredit` checks for zero address on user, but this is always `msg.sender`.

Similarly, `_removeCredit` verifies that user credit is greater or equal to the amount to remove, but `_removeCredit` is only called by `executeRedemptions` and `ltAmount_` is equal to the user credit. Therefore, `_removeCredit` validation will never fail.

RECOMMENDATION

Remove the unnecessary validations.

RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/131>

INFORMATIONAL **I-03**

ACKNOWLEDGED

Leverage Truncated To Integer

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/Factory.sol#L90
REVIEW	Remediation Review

— DESCRIPTION

Name and symbol format leverage using `targetLeverage_ / 1e18` with integer division, dropping fractional parts (e.g., 1.5x shows as 1x). This can mislead users about the product they hold.

— RECOMMENDATION

Be aware to only deploy leveraged tokens without fractional leverage amounts.

— RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/127>

INFORMATIONAL **I-04**

ACKNOWLEDGED

Unused Events

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/interfaces/IGlobalStorage.sol#L40-L42
REVIEW	Remediation Review

DESCRIPTION

`IGlobalStorage` declares an event `SetPerpOwnerHoldsFundsThreshold` but it's never used. Additionally the `SetReferreeRebate` event is duplicated.

RECOMMENDATION

Remove duplicate and unused events.

RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/126>

INFORMATIONAL **I-05**

ACKNOWLEDGED

Referee Typo

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/GlobalStorage.sol#L46
REVIEW	Remediation Review

DESCRIPTION

The parameter and state names use `referree` instead of `referee` (e.g., `referreeRebate`). This typo also appears on `SetReferreeRebate` event and `setReferreeRebate` function.

RECOMMENDATION

Fix typos.

RESOLUTION

Status: Acknowledged.

<https://github.com/bounce-tech/bounce-contracts/pull/125>

INFORMATIONAL I-06

ACKNOWLEDGED

Leveraged Token Setup In HyperCore

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/bounce-contracts-team1-1759687085775/blob/main/src/LeveragedToken.sol
REVIEW	Remediation Review

DESCRIPTION

The following steps should be considered when deploying a leveraged token on hyperliquid:

- When LT contract is deployed, it's not activated. To do so, a small amount of USDC needs to be transferred to the LT address on Core. Any action done from the EVM side (i.e. adding an API wallet) will revert until the account is activated.
- Be aware that raw actions sent from EVM are only treated as requests. When executed in HyperCore, these might never succeed. Therefore, contract can't assume that actions like transferring from spot to perps or adding agent wallets, will succeed in HyperCore.
- Bridging USDT from HyperCore to HyperEMV requires a HYPE fee, so the `LeveragedToken` contract must hold some HYPE for this action.

RECOMMENDATION

Be sure to use the `/info` endpoint with `type: userRole` in order to verify that the leveraged token has `role:user` and it's `activated`, after sending some USDC in spot. Additionally, make sure to fund the `LeveragedToken` with HYPE to be able to pay for the bridge fee.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.